

A PROJECT REPORT ON

**AI ENHANCED INVOICE PROCESSING AND
PREDICTIVE ANALYSIS**

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Yadhresh Gangurde
Akash Harale
Satvik Doshi
Aditya Durge

Exam No: B400050395
Exam No: B400050410
Exam No: B400050386
Exam No: B400050387



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

AI ENHANCED INVOICE PROCESSING AND PREDICTIVE ANALYSIS

Submitted by

Yadhresh Gangurde

Exam No: B400050395

Akash Harale

Exam No: B400050410

Satvik Doshi

Exam No: B400050386

Aditya Durge

Exam No: B400050387

are bonafide students of this institute and the work has been carried out by them under the supervision of **Dr. A. G. Phakatkar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. A. G. Phakatkar

Internal Guide

Dept. of Computer Engg.

Dr. G. V. Kale

Head

Dept. of Computer Engg.

Dr. S. T. Gandhe

Principal

Pune Institute of Computer Technology

Place: Pune

Date: May 15, 2025

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**AI ENHANCED INVOICE PROCESSING AND PREDICTIVE ANALYSIS**’.*

*We would like to take this opportunity to thank **Dr. A. G. Phakatkar** for giving us all the help and guidance we needed. We are really grateful to her for her kind support. Her valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.*

*In the end our special thanks to our honorable **Dr. P. T. Kulkarni**, Principal, PICT for their continued support and valuable inputs during ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Yadhresh Gangurde
Akash Harale
Satvik Doshi
Aditya Durge
(B.E. Computer Engg.)

ABSTRACT

This project presents an AI-Enhanced Invoice Processing and Predictive Analysis system designed to streamline invoice management and generate actionable business insights. The system accepts structured, semi-structured, and unstructured invoices, converting them into a standardized format using the Gemini Flash 1.5 LLM API for document scanning and data extraction. It supports multilingual semantic search and ambiguous product identification using Cohere's embed-multilingual-v3.0 model.

Post extraction, the structured data undergoes advanced analysis: SARIMAX is used for forecasting sales and quantity trends, K-Means clustering and RFM analysis segment customers based on behavior, and the Apriori algorithm uncovers product associations through market basket analysis. All invoice and product data is stored in MongoDB, ensuring efficient data handling and retrieval.

The frontend, built with React and Material UI, offers an intuitive interface for users to upload and interact with invoice data. The system is trained and tested on the UK Online Retail Dataset (2009–2011), comprising over 42,403 invoices and 3,589 products, enabling robust performance and scalability for real-world business use.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem solving	4
2	Literature Survey	5
3	Software Requirements Specification	7
3.1	Assumptions and Dependencies	8
3.2	Functional Requirements	8
3.2.1	System Feature 1: Automated Invoice Data Extraction	8
3.2.2	System Feature 2: Semantic Product Identification	8
3.2.3	System Feature 3: Sales Forecasting	8
3.2.4	System Feature 4: Customer Segmentation	9
3.2.5	System Feature 5: Market Basket Analysis	9
3.3	External Interface Requirements	9
3.3.1	User Interfaces	9
3.3.2	Hardware Interfaces	9
3.3.3	Software Interfaces	10
3.4	Nonfunctional Requirements	10
3.4.1	Performance Requirements	10
3.4.2	Safety Requirements & Security Requirements	10
3.5	System Requirements	10
3.5.1	Database Requirements	10
3.5.2	Software Requirements	10
3.5.3	Hardware Requirements	10
3.6	Analysis Models: Agile Methodology to be applied	11
4	System Design	12
4.1	System Architecture	13
4.2	Data Flow Diagrams	14
4.3	Use Case Diagram	16
4.4	Entity Relationship Diagram	17
4.5	Class Diagram	18
4.6	Sequence Diagram	19
4.7	Activity Diagram	19

4.8	Deployment Diagram	21
5	Project Plan	22
5.1	Project Estimate	23
5.1.1	Reconciled Estimates	23
5.1.2	Project Resources	23
5.2	Risk Management	24
5.2.1	Risk Identification	24
5.2.2	Risk Analysis	25
5.3	Project Schedule	26
5.3.1	Project Task Set	26
5.3.2	Timeline Chart	27
5.4	Team Organization	28
5.4.1	Team structure	28
5.4.2	Management reporting and communication	28
6	Project Implementation	29
6.1	Overview of Project Modules	30
6.2	Tools and Technologies Used	30
6.3	Algorithm	31
6.3.1	End-to-End Algorithm of AI-Enhanced Invoice Processing and Predictive Analytics System	31
7	Software Testing	33
7.1	Types of Testing	34
7.2	Test cases & Test Results	34
8	Results	36
8.1	Outcomes	37
8.2	Screen Shots	37
9	Conclusions	42
9.1	Conclusion	43
9.2	Future Work	43
9.3	Applications	43
10	Appendix	45
10.1	Appendix A: Problem Statement Feasibility Assessment	46
10.2	Appendix B: Paper Publication Details	47
10.3	Appendix C: Plagiarism Report	49
11	References	50

List of Figures

4.1	System Architecture	13
4.2	DFD -0	14
4.3	DFD Level 1 – Detailed View	15
4.4	Use Case Diagram	16
4.5	Entity Relationship Diagram	17
4.6	Class Diagram	18
4.7	Sequence Diagram	19
4.8	Activity Diagram	20
4.9	Deployment Diagram	21
5.1	Summarized Timeline Chart	27
8.1	Dashboard	37
8.2	Product Page	38
8.3	Product Listing Page	38
8.4	Upload Invoice	39
8.5	Invoice Details	39
8.6	Analytics Overview	40
8.7	Analysis	40
8.8	Process Analysis 1	41
8.9	Process Analysis 2	41
10.1	Publication Certificate	48
10.2	Publication Certificate	49

CHAPTER 1

INTRODUCTION

1.1 Overview

In the field of automated document processing, the extraction of information from invoices and bills has become increasingly vital. However, challenges such as background noise and image quality can significantly hinder performance, particularly in real-world scenarios. Our project, “Automated Invoice Processing,” aims to address these challenges through the implementation of advanced techniques for image capture and intelligent data extraction. By employing OpenCV for image preprocessing and EasyOCR for initial text recognition, combined with the Gemini Flash 1.5 API for LLM-based data extraction, our system enhances the accuracy of text retrieval even from noisy invoice images.

To further address ambiguities in product identification, we have integrated semantic search capabilities using Cohere’s multilingual embedding model (embed-multilingual-v3.0), enabling context-aware matching and retrieval of product data. Beyond data extraction, the project incorporates predictive analytics using SARIMAX for sales and quantity forecasting, as well as K-Means clustering with RFM analysis for effective customer segmentation. Moreover, we employ Apriori algorithm for market basket analysis, helping identify co-purchase patterns in retail data.

All structured information is stored in MongoDB, ensuring efficient storage and retrieval. The entire system has been developed with a modern React and Material UI frontend, providing a seamless user experience. Our models and insights are derived using the UK Online Retail Dataset (2009–2011) available on Kaggle, which provides a rich base for real-world invoice and transaction analysis.

This project not only streamlines the handling of financial documents but also delivers valuable business insights through advanced analytics—contributing significantly to applications in financial technology, customer intelligence, and retail automation.

1.2 Motivation

The rapid digital transformation across industries has led to an exponential increase in the volume of invoices and financial documents generated daily. However, the traditional manual processes for extracting and managing this information are often time-consuming and prone to human error. Personal experiences with the challenges of maintaining accurate records and the inefficiencies of manual data entry have motivated the development of this project.

By automating the extraction of information from invoices and bills, we aim to streamline operations and enhance accuracy, allowing businesses to focus on more strategic tasks. Furthermore, the growing prevalence of noisy environments, particularly in settings where invoices are handled, poses significant challenges to data quality. This project seeks to address these issues by leveraging advanced image processing and natural language processing techniques to improve the reliability of data extraction. Ultimately, our goal is to enhance the efficiency of financial operations, leading to better decision-making and improved user experiences in various applications.

1.3 Problem Definition and Objectives

Problem Definition:

Design and develop an AI-powered invoice processing and business analytics system that

automates the extraction of key information from scanned invoices using advanced OCR and LLM techniques. The system will further enhance business intelligence through semantic product identification, customer segmentation, demand forecasting, and association analysis. A modern web interface will facilitate smooth user interaction, and all data will be stored in a centralized NoSQL database for further processing and reporting.

Objectives:

1. **Automated Invoice Processing:** Implement OpenCV for image enhancement and Gemini Flash 1.5 API for extracting structured information from scanned invoices.
2. **Semantic Product Identification:** Integrate Cohere's `embed-multilingual-v3.0` model to resolve ambiguous product names through multilingual semantic search.
3. **Sales and Quantity Forecasting:** Use SARIMAX (Seasonal ARIMA with exogenous variables) to predict future sales and quantity trends for inventory planning.
4. **Customer Segmentation:** Apply K-Means clustering and RFM (Recency, Frequency, Monetary) analysis to identify valuable customer segments for targeted marketing.
5. **Market Basket Analysis:** Leverage the Apriori algorithm to uncover frequent itemsets and generate association rules that suggest co-purchased products.
6. **Centralized NoSQL Data Management:** Store invoices, extracted data, and analytical insights in MongoDB to ensure scalability, accessibility, and efficient querying.
7. **User-Friendly Frontend Interface:** Build an interactive web application using React and Material UI to simplify invoice uploading, review, and analysis.

1.4 Project Scope & Limitations

Project Scope:

The AI Enhanced Invoice Processing and Predictive Analysis system automates the extraction of structured information from invoice documents using advanced OCR and large language models. It enables semantic product identification using Cohere embeddings, forecasts sales trends using SARIMAX, and segments customers with K-Means and RFM analysis. Additionally, it uncovers buying patterns through market basket analysis using the Apriori algorithm. All data is stored in MongoDB for efficient retrieval, and a React + Material UI frontend ensures a smooth user experience for uploading invoices and viewing analytics. The system is designed for use in finance, retail, and logistics domains to improve document handling and decision-making processes.

Limitations:

The AI Enhanced Invoice Processing system has a few limitations. The performance of OCR and data extraction may degrade with extremely poor-quality or highly unstructured invoice images. Semantic product matching depends on the completeness and consistency of product descriptions across invoices. SARIMAX forecasting may be less effective on short or irregular time series data. Customer segmentation and market basket analysis require a sufficient volume of historical data to generate meaningful insights. Moreover, since the system relies on cloud APIs (e.g., Gemini Flash, Cohere), it depends on stable internet connectivity and may incur additional API costs. Finally, while the React interface is user-friendly, users with limited technical knowledge may require brief onboarding for optimal usage.

1.5 Methodologies of Problem solving

The problem-solving methodology for this project follows a structured, iterative approach to address the challenges in traditional invoice handling and business analytics. The first step involves identifying key problems such as inefficient manual data entry, ambiguity in product information, and lack of predictive insights in business operations. Through a study of retail and finance workflows, these challenges are clearly defined and prioritized. A detailed analysis of existing document processing and analytics techniques is conducted to highlight gaps in accuracy, scalability, and real-time decision-making. This analysis forms the foundation for the system's design and feature development. Once the problems are well understood, the team begins gathering requirements, focusing on core features such as automated text extraction, semantic product identification, forecasting models, and actionable visual analytics.

After gathering requirements, the design and development phase begins with a strong emphasis on modularity and scalability. The system architecture is designed to independently develop and integrate modules such as OCR-based extraction, semantic search, customer segmentation, and forecasting. For example, OpenCV is used for image preprocessing, EasyOCR for initial text recognition, and Gemini Flash 1.5 API for extracting structured information from invoices. Cohere embeddings are then utilized to resolve ambiguous product names through multilingual semantic search.

Development is carried out iteratively, with each component undergoing testing and refinement. For instance, the SARIMAX model for sales forecasting is trained and validated using historical data from the UK Online Retail dataset, while K-Means clustering with RFM analysis segments customers based on behavior. The Apriori algorithm is implemented to analyze product co-purchase patterns. All data is stored in MongoDB for seamless access and scalability.

The React-based frontend, built with Material UI, is developed to ensure an intuitive and responsive user interface. Once the complete system is integrated, it is tested in a controlled environment using real-world datasets. Feedback from users and domain experts is continuously incorporated to fine-tune system performance. This iterative approach ensures that the application is robust, adaptable, and capable of meeting the evolving needs of invoice-driven analytics in business environments.

CHAPTER 2

LITERATURE SURVEY

Several research efforts have focused on developing effective techniques for automating invoice processing and enhancing data extraction accuracy from diverse invoice formats. This survey provides an overview of significant studies exploring approaches ranging from machine learning algorithms to rule-based methods, all aimed at improving the efficiency, reliability, and adaptability of invoice processing systems. These studies highlight challenges in real-world scenarios and present innovative solutions contributing to advancements in automated invoice management.

An overview of Named Entity Recognition (NER) methods emphasized various techniques and their applications in natural language processing [1]. The study categorized NER approaches into rule-based, statistical, and deep learning methods, analyzing their strengths and weaknesses, and underscored the importance of context and linguistic features in improving NER accuracy.

A comparative analysis of EasyOCR and TesseractOCR for Automatic License Plate Recognition (ALPR) using deep learning algorithms evaluated both OCR tools in terms of accuracy, speed, and efficiency [2]. The findings suggested that EasyOCR outperformed Tesseract in various scenarios, particularly in recognition accuracy and adaptability to different lighting conditions.

A novel technique for handwritten text recognition using EasyOCR focused on enhancing recognition accuracy through data augmentation and model fine-tuning [3]. The method significantly improved EasyOCR's performance on diverse handwritten datasets, showcasing its applicability in real-world scenarios requiring accurate text extraction.

An enhanced number plate recognition system for restricted area access control integrated deep learning models with EasyOCR [4]. The multi-stage recognition pipeline reduced false positives and increased recognition speed, demonstrating effectiveness in real-time security applications.

A survey on BERT (Bidirectional Encoder Representations from Transformers) and its applications in natural language processing covered its architecture, pre-training, and fine-tuning processes [5]. The study highlighted BERT's versatility and effectiveness as a foundational model for NLP advancements.

Shape matching and object recognition using low distortion correspondence methods leveraged geometric properties to enhance recognition accuracy in complex visual environments [6]. The proposed approach significantly improved performance compared to traditional methods.

The Multi-Layout Invoice Document Dataset (MIDD) was introduced for named entity recognition tasks in invoice processing [7]. This dataset, with diverse invoice layouts, supports the training and evaluation of NER models, advancing invoice processing automation.

Insights into Optical Character Recognition (OCR) technology detailed its evolution, techniques, and applications [8]. The study discussed challenges like font variations and proposed solutions to enhance recognition accuracy.

A survey on text line segmentation of historical documents reviewed various segmentation techniques and their effectiveness across document types [9]. It highlighted the need for specialized approaches to preserve historical context while improving text extraction accuracy.

An approach for character recognition using document analysis with OCR integrated document analysis techniques to improve text recognition rates [10]. The method demonstrated significant improvements in recognition accuracy compared to traditional OCR approaches.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

Assumptions:

1. **Stable Network Connection:** Users are expected to have reliable internet connectivity for uploading invoices and interacting with APIs.
2. **Modern Computer Systems:** Users will use systems capable of running a modern browser and handling basic machine learning outputs.
3. **User Familiarity with Business Data:** It is assumed that users have basic knowledge of invoice documents and sales analytics.

Dependencies:

1. **Web Technologies:** Reliance on React, Material UI, and backend services for processing and displaying data.
2. **External APIs:** Usage of Gemini Flash 1.5 (LLM API) for invoice data extraction and Cohere embeddings for semantic search.
3. **Database Integration:** Dependency on MongoDB for structured data storage and retrieval.
4. **Python Libraries:** Use of machine learning and data analytics libraries like statsmodels, scikit-learn, and mlxtend.

3.2 Functional Requirements

3.2.1 System Feature 1: Automated Invoice Data Extraction

Description:

The system must enable users to upload invoice images or PDFs and automatically extract relevant structured data.

Requirements:

1. Users can upload scanned invoices or digital PDFs.
2. The system must extract fields such as invoice number, date, products, quantity, and total using Gemini Flash API.
3. Extracted data must be displayed and stored in the MongoDB database.

3.2.2 System Feature 2: Semantic Product Identification

Description:

Resolve ambiguity in product names using semantic similarity techniques.

Requirements:

1. The system must use Cohere's multilingual embeddings to match and identify product entries.
2. Users can search products using natural language queries.
3. System should support multilingual descriptions.

3.2.3 System Feature 3: Sales Forecasting

Description:

The platform must predict future sales trends using historical invoice data.

Requirements:

- 1. Use the SARIMAX model to forecast quantity and sales value.
- 2. Display forecasted results in tabular and graphical formats.
- 3. Allow users to choose forecasting range and product.

3.2.4 System Feature 4: Customer Segmentation

Description:

Group customers based on their purchasing behavior using RFM and clustering.

Requirements:

- 1. Perform RFM analysis using purchase history.
- 2. Apply KMeans clustering to categorize customer segments.
- 3. Provide insights through visual dashboards.

3.2.5 System Feature 5: Market Basket Analysis

Description:

Analyze co-purchased products to recommend combinations.

Requirements:

- 1. Implement the Apriori algorithm to discover frequent itemsets.
- 2. Display rules and confidence levels.
- 3. Visualize association results for user interpretation.

3.3 External Interface Requirements

3.3.1 User Interfaces

Feature	Description
Web Dashboard	Modern, responsive React + Material UI interface for uploading invoices and viewing analytics.
Analytics Visualizer	Interactive graphs and tables showing forecasts, segments, and market rules.

Table 3.1: User Interfaces

3.3.2 Hardware Interfaces

Hardware Component	Description
User Systems	Any desktop or laptop with a modern browser (Chrome/Firefox).
Internet Connectivity	Required to access API endpoints for invoice processing and semantic search.

Table 3.2: Hardware Interfaces

3.3.3 Software Interfaces

Software Interface	Description
MongoDB	NoSQL database for storing invoices, analytics, and metadata.
Cohere API	Embedding service for semantic product search.
Gemini Flash API	LLM-based invoice field extraction from scanned documents.

Table 3.3: Software Interfaces

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- 1. Data extraction should complete within 5 seconds per invoice on average.
- 2. Forecasting and clustering operations should return results in under 3 seconds for datasets up to 100,000 records.
- 3. The frontend must respond within 1 second to user interactions.

3.4.2 Safety Requirements & Security Requirements

- 1. All data transmitted must be encrypted using HTTPS.
- 2. Access to the system must require authenticated login.
- 3. Data backups must be performed daily.
- 4. Secure storage practices must be followed for API keys and credentials.

3.5 System Requirements

3.5.1 Database Requirements

- 1. Use MongoDB with collections for invoices, customer profiles, products, and model outputs.
- 2. Should support complex queries and analytics on large datasets.

3.5.2 Software Requirements

- 1. Frontend: React.js with Material UI
- 2. Backend: Python (Flask or FastAPI) with dependencies like Pandas, Scikit-learn, Statsmodels
- 3. APIs: Gemini Flash 1.5, Cohere
- 4. Dev Tools: Postman, Jupyter Notebook, GitHub

3.5.3 Hardware Requirements

- 1. Developer Machines: Minimum 8 GB RAM, i5 processor or equivalent
- 2. User Machines: Any system with modern browser
- 3. Hosting: Cloud server with GPU support preferred for model training

3.6 Analysis Models: Agile Methodology to be applied

The Agile methodology will guide the development of the AI Enhanced Invoice Processing project, emphasizing iterative development, collaboration, and responsiveness to user feedback:

1. **Iterative Development:** Implement the system in short sprints focusing on specific modules (OCR, search, forecasting, etc).
2. **Collaboration:** Regular meetings with mentors and domain experts to ensure alignment with goals.
3. **Adaptive Planning:** Continuously revise features based on real-time feedback from testing.
4. **Continuous Testing:** Perform unit and integration testing at each development cycle.
5. **Customer Feedback:** Collect feedback from stakeholders on usability and functionality for incremental improvements.

This adaptive methodology ensures continuous evolution of the product to meet dynamic needs in financial document automation and analysis.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

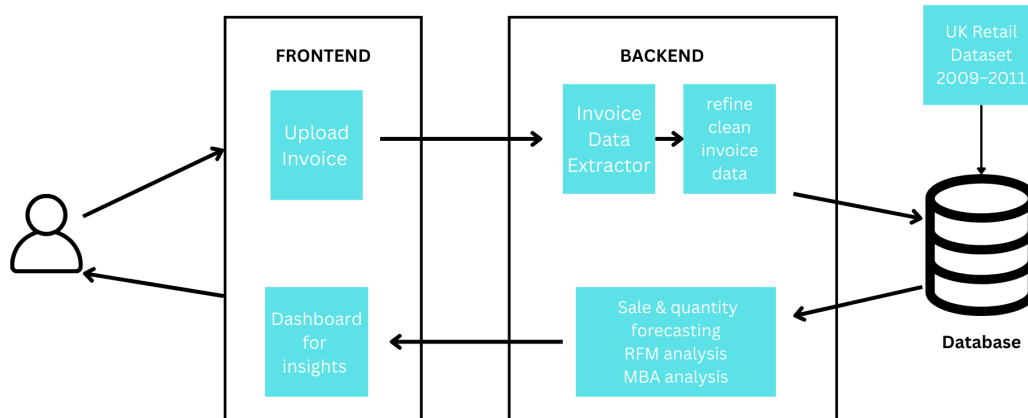


Figure 4.1: System Architecture

The system architecture involves various components as described below:

1. Users (Invoice Submitter, System Admin):

Users interact with the system through a user interface.

- **Invoice Submitters** upload invoice documents.
- **System Admins** monitor the process and manage configurations.

2. Frontend (React + Material UI):

The web-based UI is built using React and Material UI.

- Users can upload invoices via this interface.
- The UI enables smooth interaction and upload functionality.

3. Backend (FastAPI):

FastAPI handles communication between the frontend and AI/ML services.

- Receives uploaded invoices and orchestrates processing.
- Handles database interactions for storing and retrieving data.

4. AI/ML Services:

Various AI services are used to extract, process, and analyze invoice data:

- **LLM Invoice Extraction (Gemini-flash-1.5):** Extracts structured data from invoices.
- **Semantic Search (Cohere):** Resolves ambiguous or multilingual product IDs using embeddings.

- **Sales & Quantity Forecasting (SARIMAX):** Predicts future sales based on historical trends.
- **Customer Segmentation (KMeans, RFM):** Clusters customers based on behavior and value.
- **Market Basket Analysis (Apriori):** Identifies product association rules and frequent itemsets.

5. MongoDB (Invoices, Products, Analytics):

A centralized NoSQL database stores all relevant data:

- Structured invoice data.
- Product embeddings.
- Forecasts, customer segments, and basket rules.

6. Dataset Loader (UK Retail Dataset 2009–2011):

An initial dataset is loaded into the system for training and testing purposes.

- Contains historical sales data from UK retail.
- Serves as baseline data for forecasting and benchmarking.

7. Analytics, Monitoring & Visualization:

The system supports detailed analysis and monitoring capabilities.

- Enables generation of visualizations for sales trends, customer clusters, and basket analysis.
- Assists system admins in decision-making and system optimization.

4.2 Data Flow Diagrams

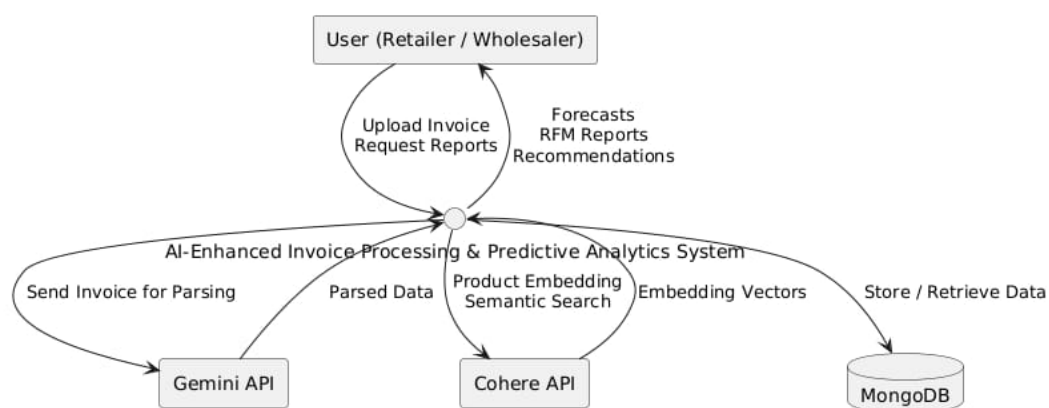


Figure 4.2: DFD -0

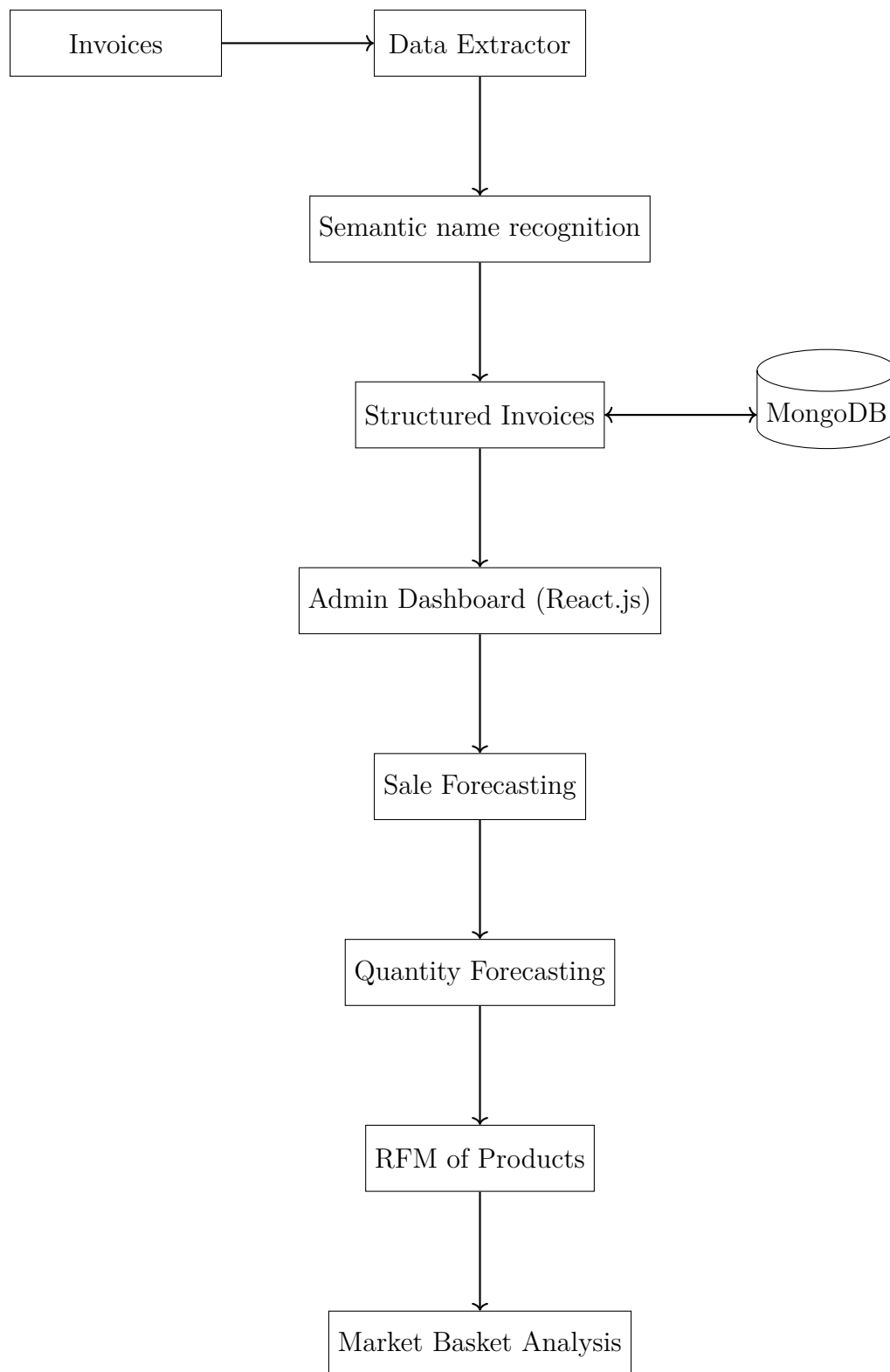


Figure 4.3: DFD Level 1 – Detailed View

4.3 Use Case Diagram

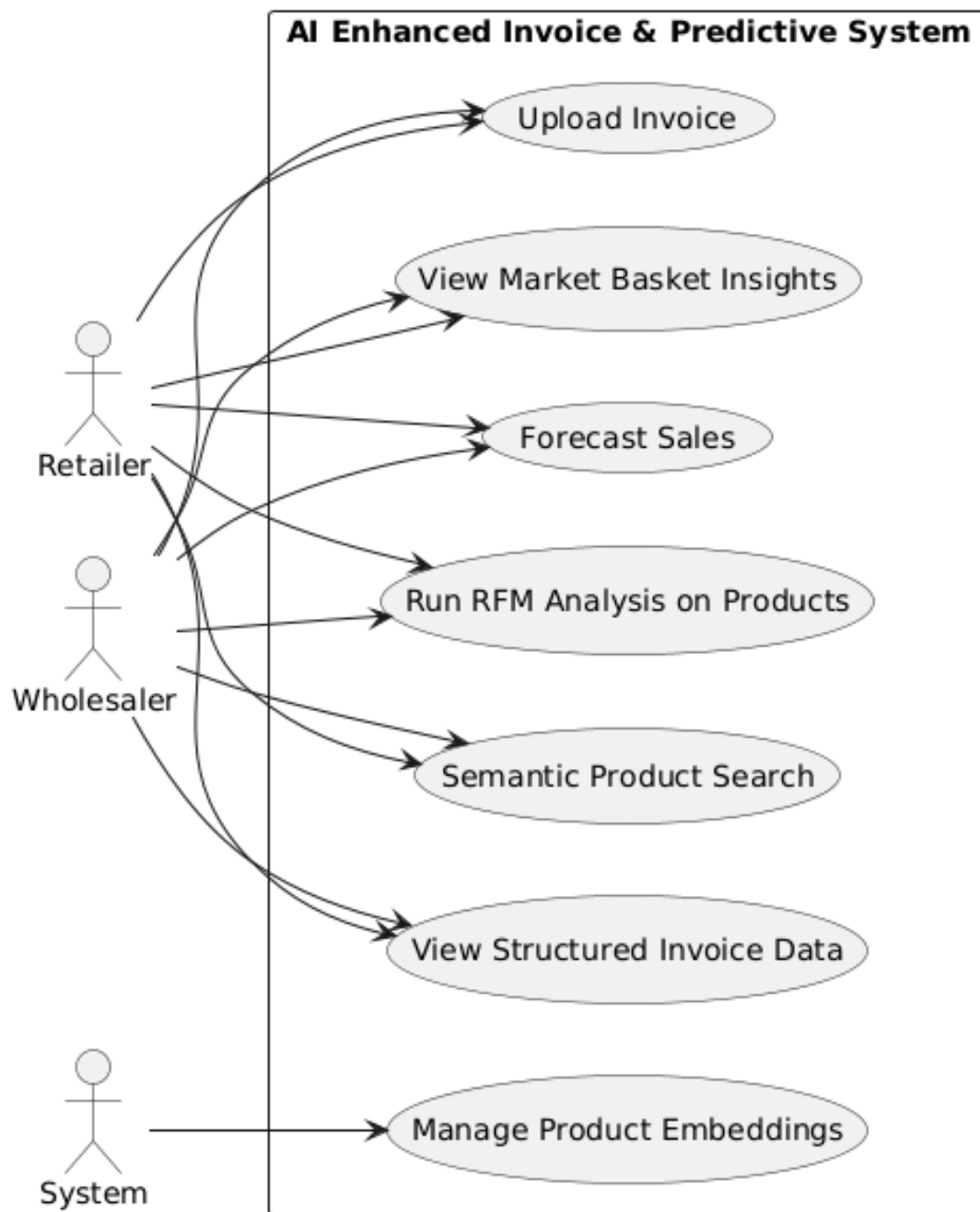


Figure 4.4: Use Case Diagram

4.4 Entity Relationship Diagram

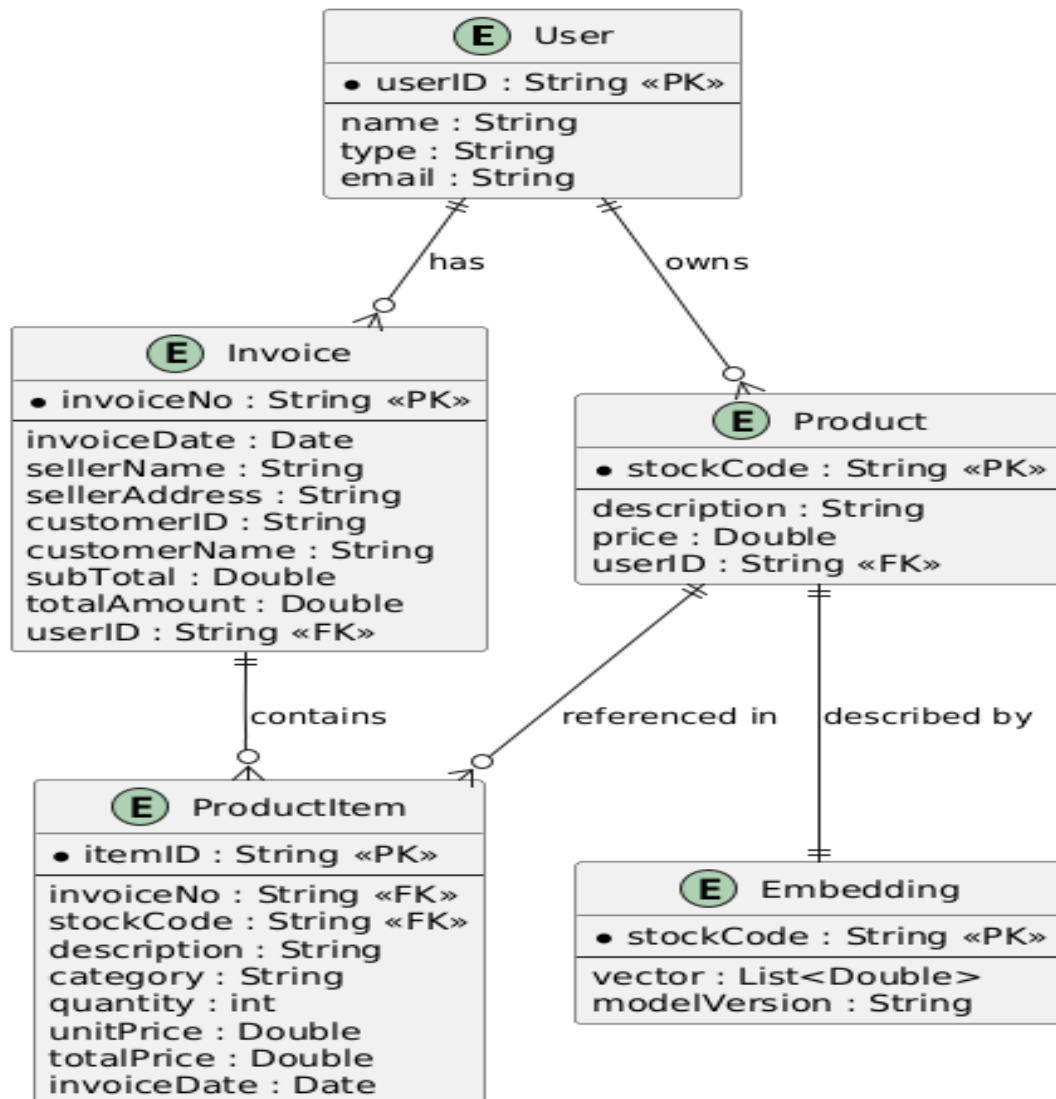


Figure 4.5: Entity Relationship Diagram

4.5 Class Diagram

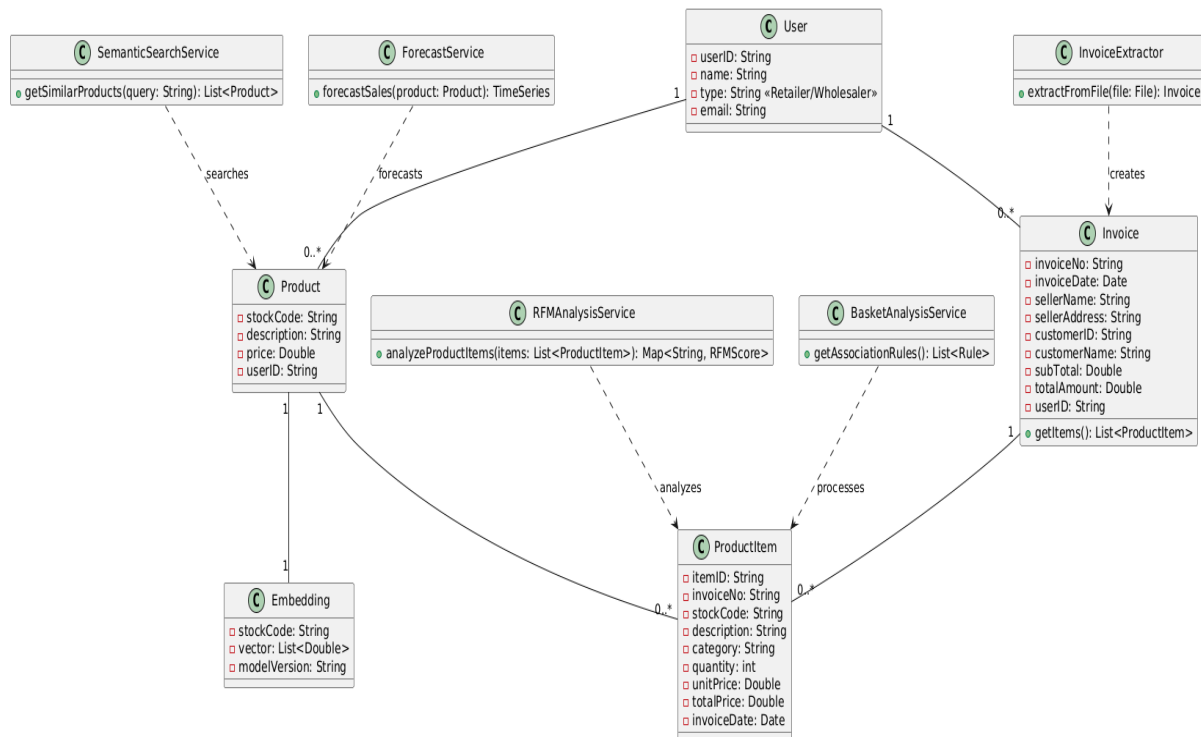


Figure 4.6: Class Diagram

4.6 Sequence Diagram

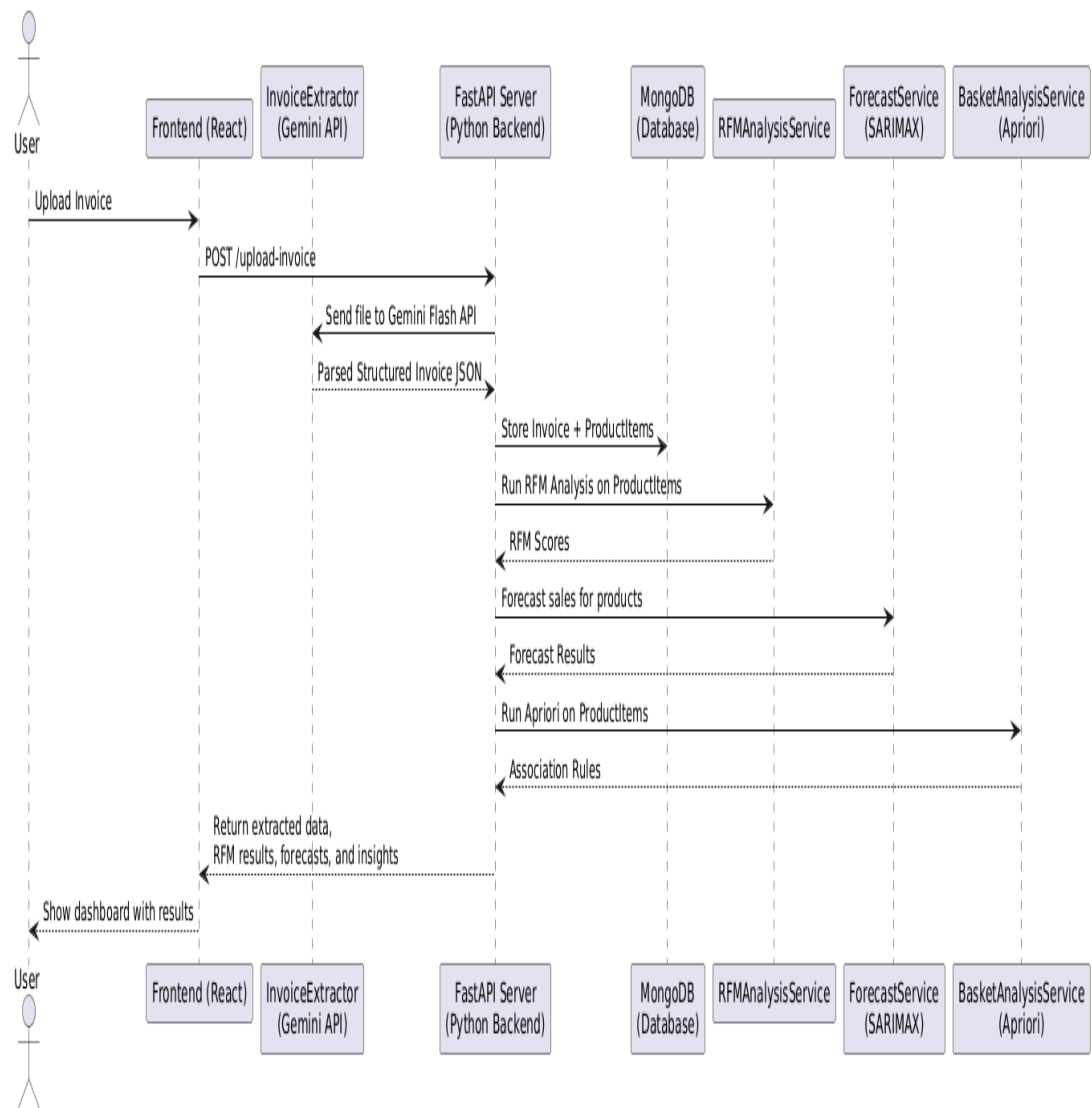


Figure 4.7: Sequence Diagram

4.7 Activity Diagram

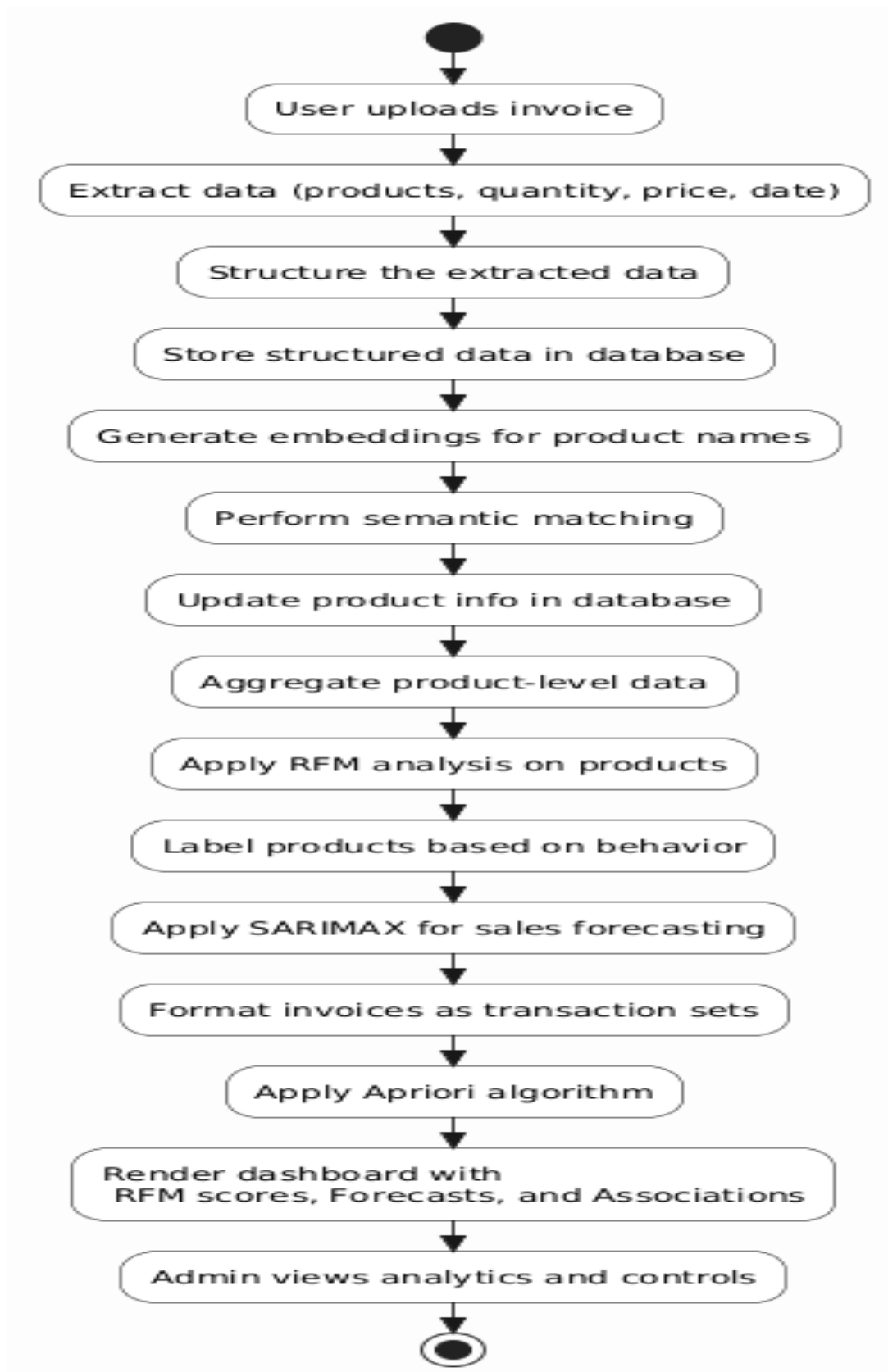


Figure 4.8: Activity Diagram

4.8 Deployment Diagram

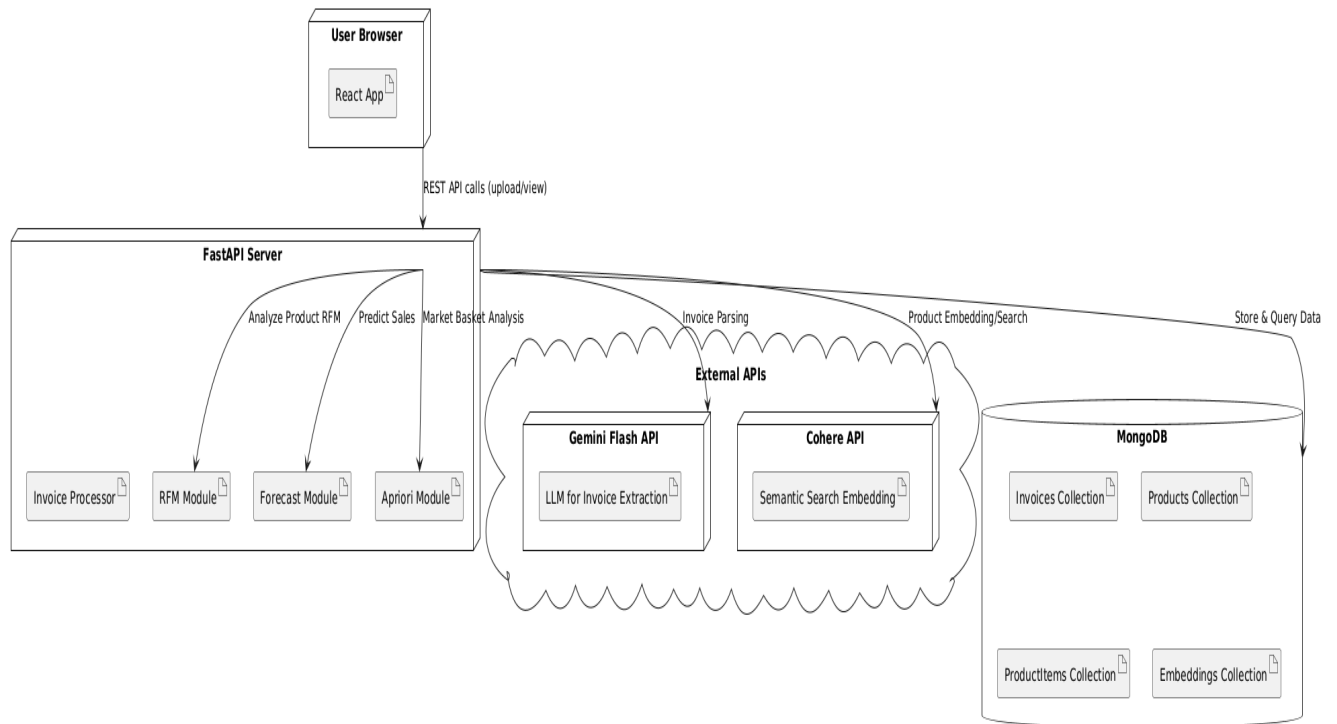


Figure 4.9: Deployment Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

1. Research and Development phase: Estimated at 6 months, focuses on building core features and developed invoice upload, scanning and extraction using Gemini Flash 1.5 API. Implemented semantic search with Cohere, sales forecasting (SARIMAX), customer segmentation (K-Means & RFM), and market basket analysis (Apriori).
2. Testing and Evaluation: Estimated at 2 months, involves performance, security, and user acceptance testing.
3. Documentation and Review: Estimated at 1.5 months, will include creating user guides, technical documentation, and feedback-driven reviews. This ensures the system's usability, accuracy, and alignment.

5.1.2 Project Resources

1. Frontend Developer

- Responsible for designing and developing the user interface using React.js and Material UI.
- Create a user-friendly dashboard for uploading invoices and visualizing analytics.
- Ensure responsive design for seamless access across devices and screen sizes.

2. Backend Developer

- Develop and maintain backend logic using Node.js and Python.
- Integrate Gemini Flash 1.5 LLM API for invoice data extraction and Cohere API for semantic search.
- Implement endpoints for forecasting, segmentation, and market basket analysis modules.

3. Database Technology

- Utilize MongoDB for storing invoice, product, and analysis data with flexible schema design.
- Implement indexing and aggregation pipelines for efficient querying and reporting.
- Ensure data integrity and scalability for handling real-world retail invoice datasets.

4. Collaboration Tools

- Use GitHub for version control and collaboration among team members.
- Conduct regular team meetings via Google Meet to coordinate development efforts.
- Manage tasks and milestones using Trello or Jira for organized project tracking.

5.2 Risk Management

Risk management is crucial for the successful implementation of the AI-Enhanced Invoice Processing and Predictive Analysis system. Identifying potential risks early helps in minimizing disruptions and ensuring reliable system deployment.

1. **API Integration Risks:** Issues may occur while integrating external APIs such as Gemini Flash 1.5 and Cohere. These will be mitigated through extensive testing, fallback mechanisms, and regular monitoring.
2. **Data Quality and Extraction Risks:** Inconsistent invoice formats may affect data extraction accuracy. We will handle this by implementing preprocessing steps and validating extracted fields before analysis.
3. **Scalability and Performance Risks:** Processing large volumes of invoice data may lead to performance bottlenecks. The system will be optimized using efficient MongoDB queries and scalable backend architecture.
4. **User Experience Risks:** A complex interface may hinder usability. We will conduct user testing and incorporate feedback to ensure a seamless and intuitive frontend experience.
5. **Timeline Risks:** Delays in model training or integration could affect overall delivery. An agile development approach with regular sprint reviews will be adopted to address setbacks early.

By actively managing these risks, we aim to ensure the robustness, usability, and successful deployment of the system for real-world business applications.

5.2.1 Risk Identification

Risk identification is crucial for anticipating potential challenges that could affect the development and deployment of the AI-Enhanced Invoice Processing and Predictive Analysis system. The following risks have been identified:

- **API Integration Risks:** Integrating third-party APIs such as Gemini Flash 1.5 and Cohere embedding may lead to compatibility or response latency issues.
- **Data Extraction Accuracy Risks:** Variability in invoice formats and languages can lead to inaccurate data extraction, affecting downstream analytics.
- **Semantic Search Challenges:** Ambiguous product descriptions and multilingual data may reduce the effectiveness of semantic search and identification.
- **Forecasting and Clustering Risks:** SARIMAX and K-Means models may underperform if the dataset lacks seasonal patterns or proper segmentation features.
- **Performance Risks:** Processing large datasets, such as the UK Retail Dataset, may cause memory or response time issues without optimized queries and model execution.
- **Data Privacy and Security Risks:** Storing invoice and customer data in MongoDB requires robust encryption and access control to prevent breaches.

- **User Adoption Risks:** Complex dashboards or workflows may hinder user experience. Continuous user feedback will be vital to enhance usability.

5.2.2 Risk Analysis

Risk analysis involves evaluating the identified risks to determine their potential impact on the AI-Enhanced Invoice Processing and Predictive Analysis project and the likelihood of their occurrence. This helps prioritize mitigation strategies for effective project execution.

- **API Integration Risks:**
 - **Impact:** High – Failure in integrating Gemini Flash 1.5 or Cohere APIs could disrupt core functionality.
 - **Likelihood:** Medium – Dependencies on external services pose a moderate risk but can be managed with retries and fallbacks.
- **Data Extraction Accuracy Risks:**
 - **Impact:** High – Incorrect extraction from invoices may affect forecasting and analytics accuracy.
 - **Likelihood:** Medium – While models are robust, inconsistent formats in real-world data may lead to occasional errors.
- **Semantic Search Challenges:**
 - **Impact:** Moderate – Ambiguous product descriptions could limit search reliability.
 - **Likelihood:** Medium – Pre-trained embeddings help, but domain-specific tuning might be required.
- **Forecasting and Clustering Risks:**
 - **Impact:** Moderate – Model inaccuracies could lead to incorrect trend and customer segmentation insights.
 - **Likelihood:** Medium – Proper validation and tuning can reduce these risks.
- **Performance Risks:**
 - **Impact:** High – Processing large datasets like the UK Retail dataset may cause latency or crashes.
 - **Likelihood:** Medium – Can be mitigated through indexing, query optimization, and resource scaling.
- **Data Privacy and Security Risks:**
 - **Impact:** Critical – Breaches can result in legal consequences and loss of trust.
 - **Likelihood:** Medium – Strong encryption and access control will reduce exposure, but risks remain.
- **User Adoption Risks:**

- **Impact:** Moderate – A non-intuitive UI could hinder adoption by business users.
- **Likelihood:** Medium – Usability testing and feedback collection throughout development will address this.

5.3 Project Schedule

5.3.1 Project Task Set

1. **Requirement Analysis:** Identify business challenges related to manual invoice processing, data inconsistency, and delayed insights.
2. **Stakeholder Meetings:** Conduct meetings with finance teams, procurement heads, and compliance officers to gather functional and non-functional requirements.
3. **Problem Framing and Goal Setting:** Define clear objectives for automation, semantic search, forecasting, and business insights.
4. **Technology Stack Selection:** Finalize the stack, including React.js for UI, Node.js for backend, MongoDB for storage, and APIs like Gemini Flash 1.5 or Cohere for AI.
5. **Dataset Selection:** Choose datasets such as UK Retail dataset and vendor invoice samples for model training and testing.
6. **Team Role Allocation:** Assign roles such as frontend developer, backend developer, ML engineer, and QA tester.
7. **Learning and Setup:** Team members familiarize themselves with chosen libraries (e.g., LangChain, Pinecone, Prophet) and set up the development environment.
8. **UI/UX Design:** Design intuitive dashboards and input interfaces for invoice upload, query search, and insights visualization.
9. **Design Review:** Validate design mockups with end users for usability and required modifications.
10. **Data Preprocessing and Annotation:** Clean and format invoice datasets; annotate if necessary for training models.
11. **Model Selection:** Identify appropriate models for invoice extraction (LLMs), product search (vector embedding), forecasting (Prophet), and clustering (KMeans).
12. **Model Development and Testing:** Train and test models for each functionality—fine-tune as needed for accuracy and performance.
13. **API Integration:** Integrate AI/ML functionalities into the backend via REST APIs and ensure data flow consistency.
14. **Frontend Development:** Build user-facing components to support file uploads, semantic search, data visualization, and feedback collection.

15. **System Integration:** Connect frontend, backend, and ML modules into a unified solution.
16. **Security and Compliance:** Implement encryption, access control, and compliance checks (e.g., GDPR) for sensitive financial data.
17. **Testing Phase:** Perform unit, integration, and system testing to identify and resolve defects.
18. **User Evaluation and Feedback:** Conduct UAT (User Acceptance Testing) with actual finance users to collect feedback and usability insights.
19. **Iterative Improvements:** Refine modules based on feedback and evolving requirements.
20. **Documentation:** Document architecture, code, APIs, model performance, and deployment steps.
21. **Project Submission and Handover:** Deliver the final product along with user manuals, deployment scripts, and demo walkthroughs.

5.3.2 Timeline Chart

This section represents a summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

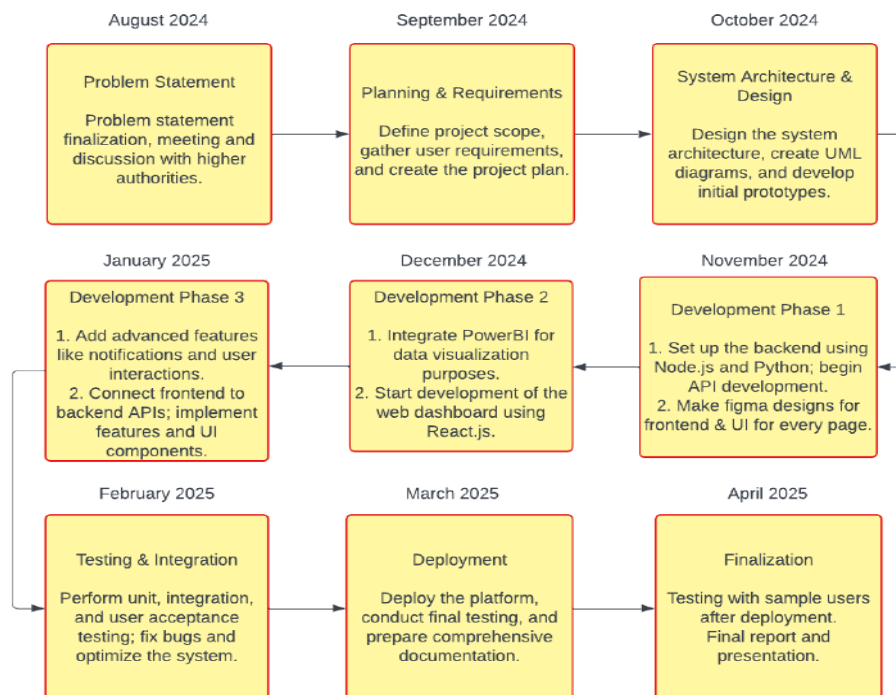


Figure 5.1: Summarized Timeline Chart

5.4 Team Organization

5.4.1 Team structure

- **Yadhresh Gangurde:** AI/ML, Model Training.
- **Akash Harale:** Frontend Developer, System Designer.
- **Satvik Doshi:** Backend Developer, Documentarian.
- **Aditya Durge:** Backend Developer, Documentarian.
- **Dr. A. G. Phakatkar:** Internal Mentor and Guide.

5.4.2 Management reporting and communication

Management

For management during our project, we used several effective tools and practices to ensure regular and timely updates. We used **Google Groups** for team communication, where we regularly shared updates, research papers, and important documents. Additionally, we collected various research papers related to our project and organized as well as structured them properly within **Google Drive**. This made it easy for the entire team to access and refer to the collected material. For code management and version control, we used **Git** and **GitHub**, ensuring that all team members could collaborate, track changes, and contribute to the project.

Reporting

Bi-weekly, we reported our progress to the internal mentor, and provided updates on work completed, challenges faced, and ideas we came up with. The regular communication allowed us to stay aligned with project goals and deadlines of submissions. We consulted our advisor **Dr. A. G. Phakatkar** for new ideas and strategic guidance, which we will be implementing in our project, ensuring improvements and innovation.

Communication

For regular communication with team members, we used **Google Groups** and **WhatsApp** for quick updates, making it easy to share new information and get immediate feedback. Our communication with the mentor was structured around bi-weekly sessions and regular updates, with tasks given accordingly.

Thus, this structured approach to management, reporting, and communication helped us to maintain clarity, accountability, and efficiency throughout the project.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The AI-Enhanced Invoice Processing and Predictive Analysis system comprises the following core modules:

- **Invoice Upload and Extraction Module:** Allows users to upload invoices in various formats (PDF, image, text) and uses AI-based parsing (via LLMs) to extract key fields such as invoice number, date, vendor name, line items, and total amounts.
- **Semantic Search Module:** Enables users to query historical invoices or financial data using natural language. It leverages vector embedding models and similarity search (e.g., via Pinecone or FAISS) for retrieving relevant documents.
- **Predictive Analytics Module:** Employs time series forecasting (using Prophet) and clustering algorithms to analyze trends, predict expenses, detect anomalies, and group customer behavior or product patterns.
- **Backend Processing Module:** Developed using Node.js and Express.js, this module handles API endpoints, routes, data validation, and business logic integration with the AI models and database.
- **Database Module:** Utilizes MongoDB to store invoice metadata, extracted content, embeddings, and user queries securely with proper indexing and schema validation.
- **Frontend Dashboard Module:** A React.js-based interface providing features like invoice upload, search bar for querying, visualization of trends, and drill-down analysis of extracted data.
- **Visualization and Reporting Module:** Uses Chart.js or similar libraries to render dynamic charts, forecasts, and KPIs such as monthly expense trends, top vendors, product categories, and customer segments.

6.2 Tools and Technologies Used

The following tools and technologies were employed to develop the Electron Eye system:

- **Frontend:** React.js – Used for creating a user-friendly dashboard where users can upload invoices, perform semantic searches, and visualize analytics.
- **Backend:** Node.js and Express.js – Manages API endpoints, integrates with AI models, and handles communication between the frontend and database.
- **Database:** MongoDB – Stores structured invoice data, extracted fields, embeddings, and user interaction logs, offering flexibility and scalability.
- **AI/ML Models:** OpenAI (LLM) APIs, Prophet (for forecasting), and clustering algorithms – Used for invoice data extraction, predictive analytics, and customer/product segmentation.

- **Visualization:** Chart.js – For rendering visual insights such as trends, KPIs, forecasts, and comparative analytics in a dynamic format.
- **Security:** JWT-based authentication, HTTPS, and encryption techniques – Ensures secure access control, data privacy, and safe communication.
- **Version Control:** GitHub – Facilitates collaborative development, version tracking, and project management.
- **Project Management and Collaboration:** Trello and Google Meet – Used for tracking development progress, assigning tasks, and team coordination meetings.

6.3 Algorithm

6.3.1 End-to-End Algorithm of AI-Enhanced Invoice Processing and Predictive Analytics System

Input: Raw invoice documents, historical retail dataset **Output:** Structured invoice data, product insights, customer segments, sales forecasts, and market basket recommendations

1. Start the system.
2. User uploads invoice documents through the system interface.
3. Verify the uploaded invoice format. If valid, pass to the extraction module.
4. Extract key fields from the invoice including invoice ID, invoice date, product name, quantity, and price using document processing methods.
5. Structure and normalize the extracted data for consistency.
6. Store the structured data into a centralized database for persistent storage and retrieval.
7. For each product name, compute semantic embeddings to handle ambiguity or multilingual entries.
8. Match each product against stored product vectors using semantic similarity to ensure consistent identification.
9. Update the database with resolved product names and their embeddings.
10. Aggregate invoice data over time for each product.
11. Compute RFM (Recency, Frequency, Monetary) scores for each product:
 - **Recency:** Time since the product was last sold
 - **Frequency:** Number of times the product has been sold
 - **Monetary:** Total sales value of the product

12. Use RFM scores to categorize products into behavioral clusters such as “Hot Selling”, “Dormant”, or “Occasional”.
13. Perform time-series forecasting on each product’s sales using historical sales data.
14. Predict future quantities and revenue trends for each product and store the results.
15. For market basket analysis, group invoice data into product item lists per transaction.
16. Apply frequent itemset mining to identify product combinations that are often purchased together.
17. Generate and rank association rules based on support, confidence, and lift values.
18. Visualize forecasts, product RFM segments, and association rules through the analytics dashboard.
19. Provide an admin view to monitor invoice uploads, extracted data accuracy, and analytics summaries.
20. End the system.

CHAPTER 7

SOFTWARE TESTING

7.1 Types of Testing

The following types of testing were conducted to ensure the robustness and reliability of the Invoice Intelligence and Predictive Analytics system:

- **Unit Testing:** Core modules like invoice parsing, semantic search embedding, forecasting models, and MongoDB data handling were individually tested for accuracy and reliability.
- **Integration Testing:** Verified data flow and integration between the React frontend, backend, MongoDB database, and external APIs like Gemini Flash and Cohere embeddings.
- **System Testing:** Conducted end-to-end testing from invoice upload to data visualization, ensuring seamless workflow and consistent behavior across modules.
- **Functional Testing:** Ensured that key features such as invoice field extraction, product recommendation, sales forecasting, and clustering were working as per defined requirements.
- **Security Testing:** Focused on API-level security, authentication, and ensuring invoice data is securely transmitted and stored using encrypted protocols.
- **Performance Testing:** Measured system behavior under high invoice upload volume, large product datasets, and rapid dashboard interactions to ensure consistent response times.
- **User Acceptance Testing (UAT):** Received feedback from business users and domain experts to validate usability, dashboard clarity, and business relevance of insights.

7.2 Test cases & Test Results

Below are representative test cases along with their expected and actual outcomes during the testing phase:

- **Test Case 1: Invoice Upload and Extraction**
 - **Input:** Upload a standard purchase invoice in PDF format.
 - **Expected Output:** All relevant fields (product name, quantity, price, date) extracted and displayed on the UI.
 - **Actual Result:** Extraction completed successfully and displayed accurately. (Pass)
- **Test Case 2: Semantic Product Search**
 - **Input:** Search for “white wireless mouse” in a multilingual query.
 - **Expected Output:** Closest matching products retrieved from the database using embeddings.

- **Actual Result:** Accurate and relevant product suggestions shown. **(Pass)**
- **Test Case 3: Sales Forecast Visualization**
 - **Input:** Select a product with sufficient historical sales data.
 - **Expected Output:** Future demand curve plotted with confidence intervals.
 - **Actual Result:** Chart.js graph displayed forecast as expected. **(Pass)**
- **Test Case 4: Clustered Customer Segments**
 - **Input:** Run RFM-based segmentation on existing customer records.
 - **Expected Output:** Distinct clusters visualized and labeled based on spending and engagement.
 - **Actual Result:** Segmentation logic executed and visual clusters rendered. **(Pass)**
- **Test Case 5: Admin Dashboard Filter Functionality**
 - **Input:** Filter results by customer ID and invoice date range.
 - **Expected Output:** Only relevant records shown in the result set.
 - **Actual Result:** Filter logic applied and correct results returned. **(Pass)**

CHAPTER 8

RESULTS

8.1 Outcomes

- The system provides an automated solution for invoice data extraction and analysis, enabling businesses to gain actionable insights from uploaded invoices using advanced AI and machine learning models.
- Real-time sales forecasting and customer segmentation help businesses optimize inventory management, improve marketing strategies, and enhance customer targeting by predicting future sales and identifying key customer groups.
- The interactive dashboard, powered by Chart.js, presents actionable visualizations for product performance, sales trends, and market basket analysis, empowering business managers with data-driven insights to make informed decisions.

8.2 Screen Shots

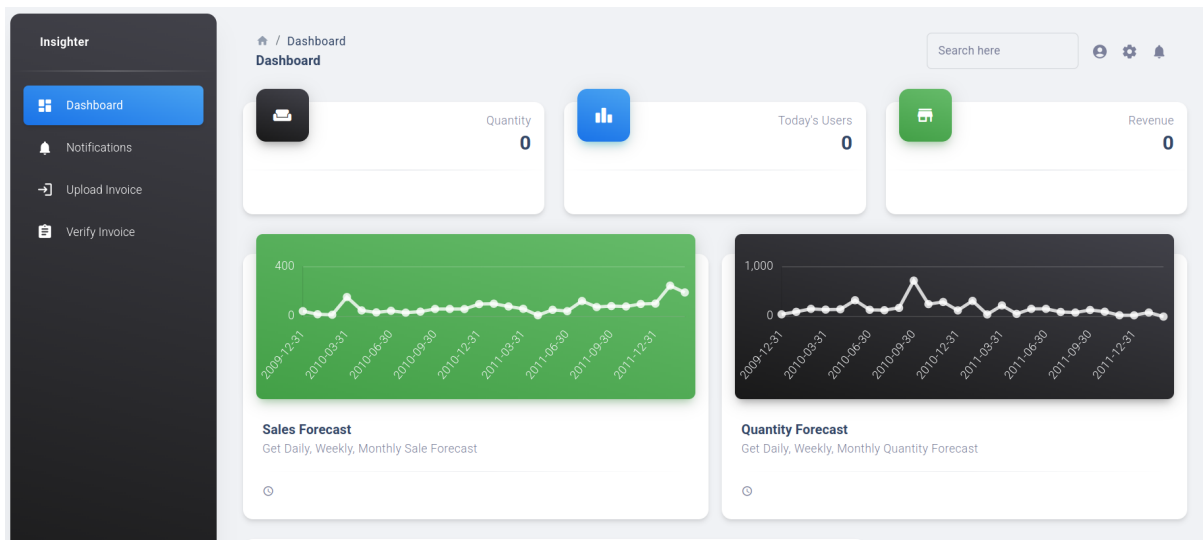


Figure 8.1: Dashboard

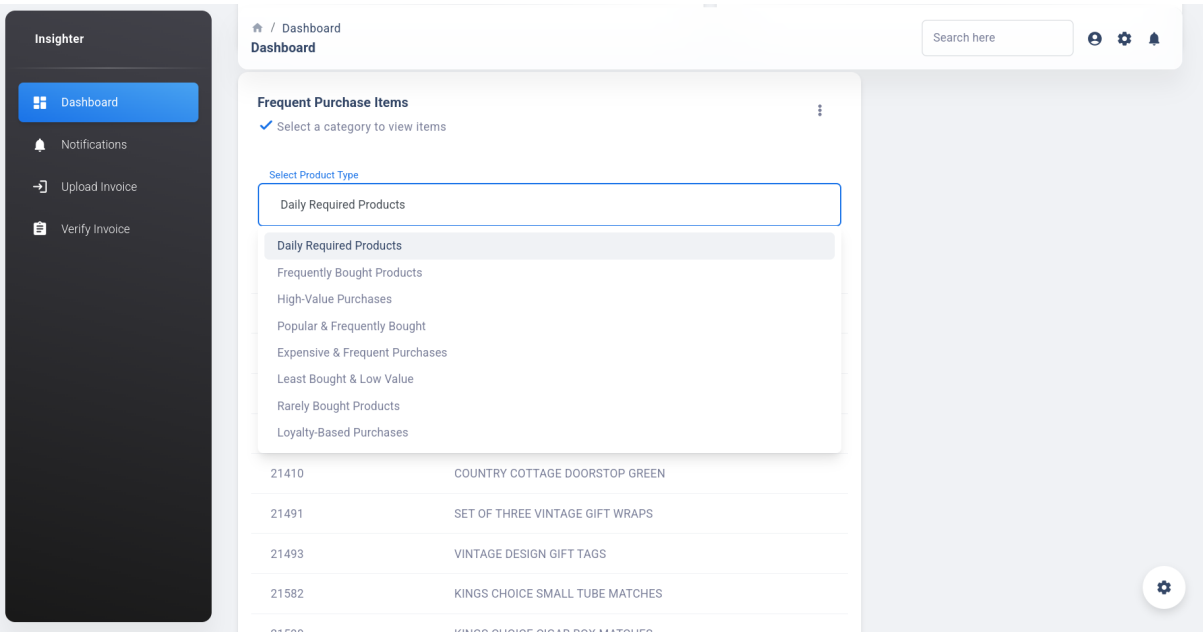


Figure 8.2: Product Page

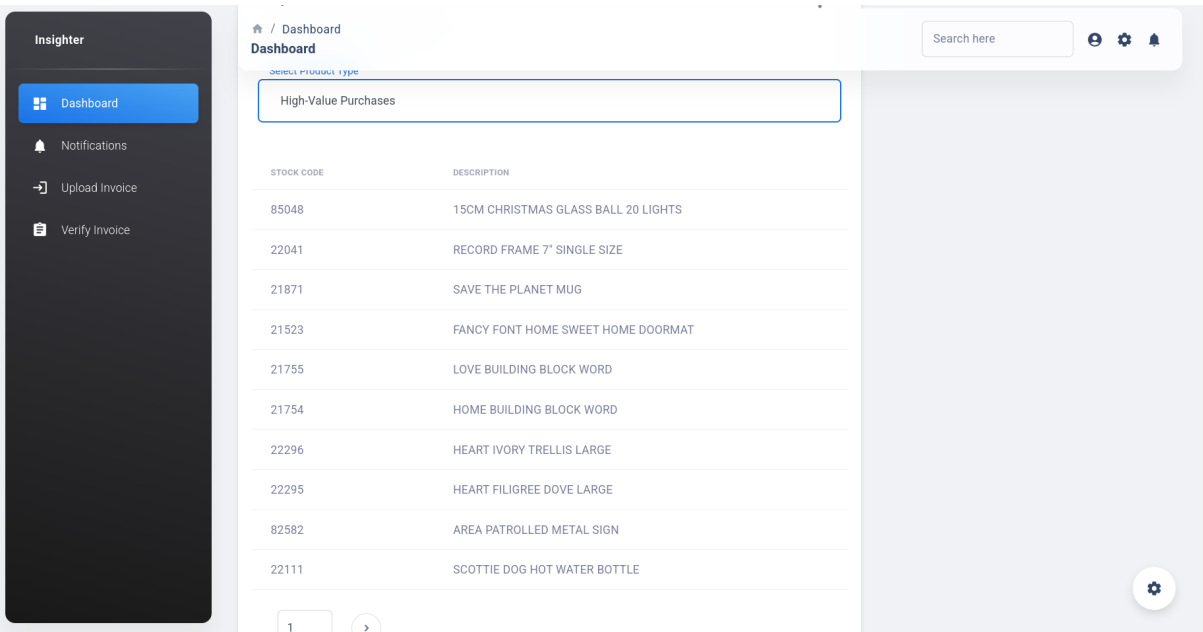


Figure 8.3: Product Listing Page

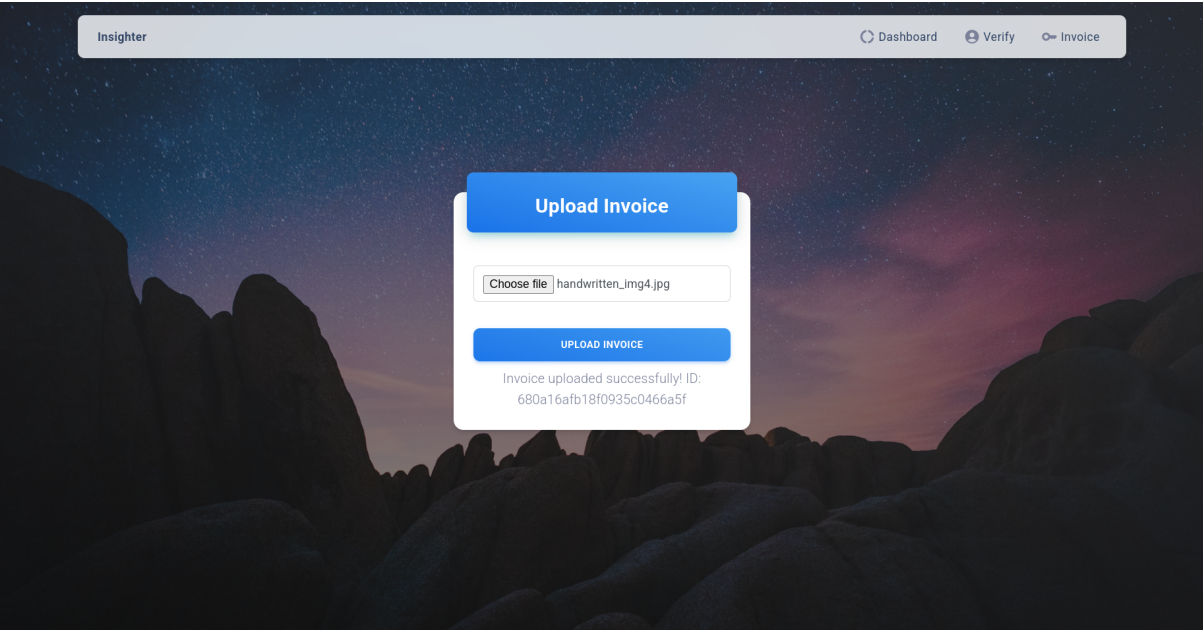


Figure 8.4: Upload Invoice

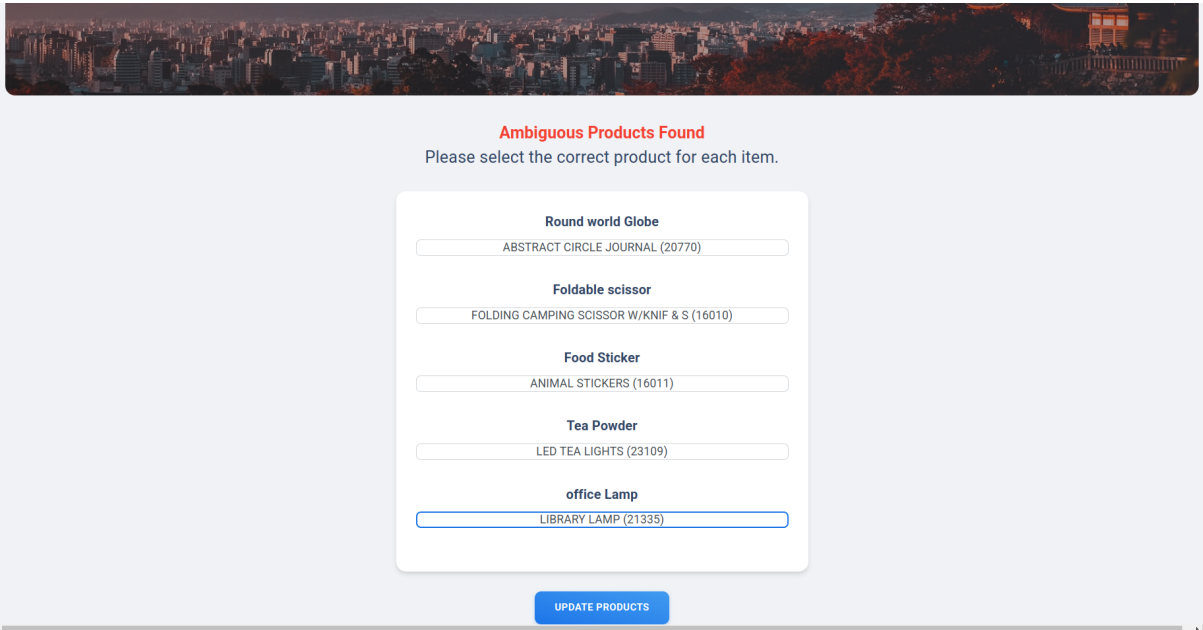


Figure 8.5: Invoice Details

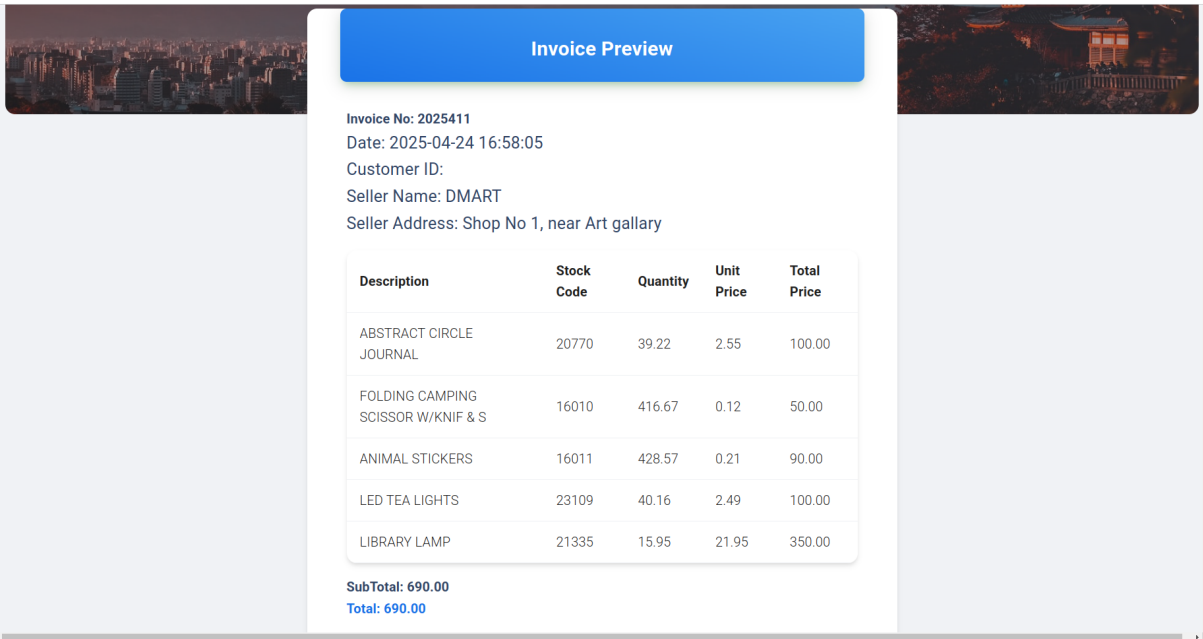


Figure 8.6: Analytics Overview

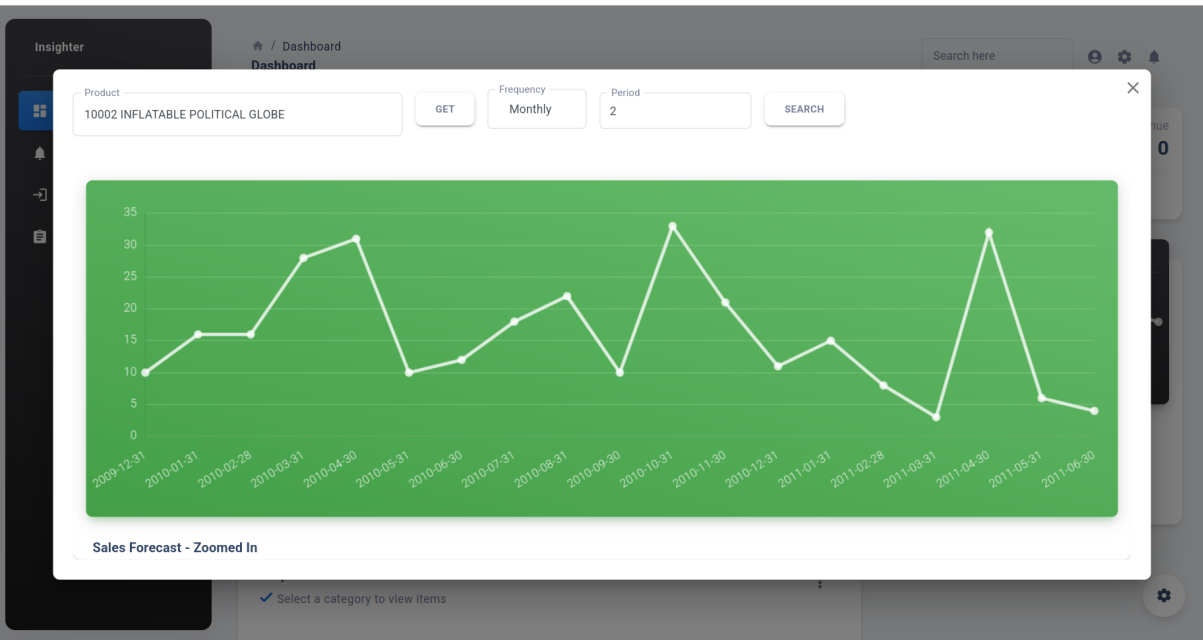


Figure 8.7: Analysis

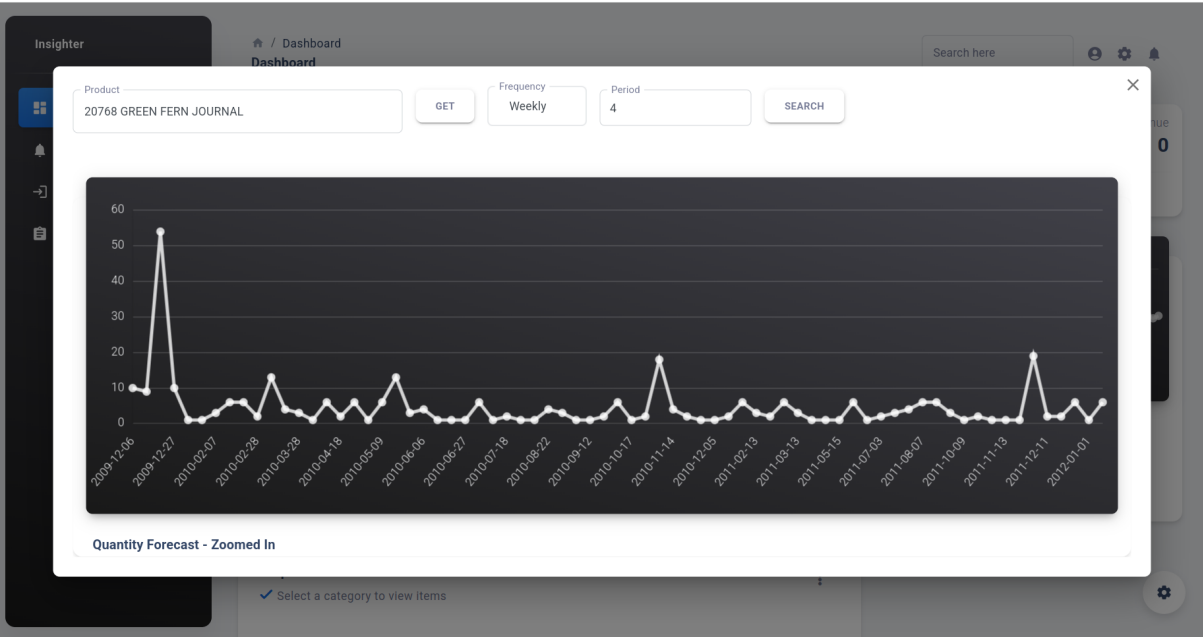


Figure 8.8: Process Analysis 1

The screenshot shows a dashboard with a sidebar menu containing 'Dashboard', 'Notifications', 'Upload Invoice', and 'Verify Invoice'. The main content area displays a table of product types under the filter 'Least Bought & Low Value'. The table has two columns: 'STOCK CODE' and 'DESCRIPTION'. The data is as follows:

STOCK CODE	DESCRIPTION
22143	CHRISTMAS CRAFT HEART DECORATIONS
22145	CHRISTMAS CRAFT HEART STOCKING
21252	SET OF MEADOW FLOWER STICKERS
21100	CHARLIE AND LOLA CHARLOTTE BAG
21410	COUNTRY COTTAGE DOORSTOP GREEN
21491	SET OF THREE VINTAGE GIFT WRAPS
21493	VINTAGE DESIGN GIFT TAGS
21582	KINGS CHOICE SMALL TUBE MATCHES
21590	KINGS CHOICE CIGAR BOX MATCHES
35951	FOLKART BAUBLE CHRISTMAS DECORATION

Figure 8.9: Process Analysis 2

CHAPTER 9

CONCLUSIONS

9.1 Conclusion

The proposed invoice intelligence and predictive analytics system will offer a cutting-edge solution to address the complexities of invoice data processing and business insights. By leveraging machine learning models for invoice extraction, sales forecasting, and customer segmentation, the system enables businesses to make data-driven decisions efficiently. It will integrate seamlessly with existing business workflows and automate critical tasks such as invoice processing, sales prediction, and market basket analysis.

With its advanced analytics capabilities, the system will empower businesses to optimize inventory, improve marketing strategies, and enhance customer targeting. The user-friendly interface will ensure that even non-technical users can easily interpret the data and make informed decisions. As businesses grow, the application will scale to accommodate expanding data and evolving business needs, making it a long-term solution for efficient and intelligent invoice and sales management.

9.2 Future Work

- **Scalability and Flexibility:** The current system focuses on the UK retail dataset and specific business processes. Future versions could incorporate more diverse datasets and features that cater to different industries and geographical regions, enabling the application to support a broader range of business needs.
- **Integration with Cloud Services:** To enhance data storage and accessibility, future development could integrate cloud solutions such as AWS or Google Cloud. This would allow businesses to store large datasets securely, access real-time insights remotely, and scale the application effortlessly as business data grows.
- **Enhanced Forecasting Models:** While the current system uses the Sarimax model for sales forecasting, future updates could incorporate more advanced machine learning algorithms like LSTM (Long Short-Term Memory) networks to improve the accuracy and precision of sales predictions, especially in volatile markets.
- **Integration with ERP Systems:** Future versions of the application could integrate with Enterprise Resource Planning (ERP) systems to provide a seamless connection between invoice processing, sales forecasting, and business resource management. This would allow businesses to automate workflows across multiple departments.
- **Mobile Application Development:** Developing a mobile application could enable business managers and sales teams to monitor real-time analytics, track invoices, and get notifications on sales trends and customer segments while on the go, increasing convenience and responsiveness.

9.3 Applications

- **Retail Businesses:** The system can be deployed by retail businesses to optimize their inventory management and sales forecasting processes. By analyzing past

sales data, businesses can predict future sales, manage stock levels, and identify high-demand products to minimize overstock and stockouts.

- **E-Commerce Platforms:** For e-commerce platforms, this application can help in identifying customer purchase patterns, segmenting customers based on purchasing behavior, and optimizing product recommendations. The predictive analytics features can also be used to forecast sales trends, helping businesses plan marketing strategies and promotions more effectively.
- **Manufacturers:** Manufacturing companies can use this system to forecast demand for their products and better manage their supply chain. By leveraging sales predictions and market basket analysis, manufacturers can ensure that they produce and distribute products that align with market demand.
- **Financial Institutions:** Financial institutions can use the application to analyze customer purchasing behavior, predict future spending patterns, and segment customers based on their financial activities. This would enable personalized financial products and marketing strategies, increasing customer retention and satisfaction.
- **Logistics and Distribution Companies:** Logistics companies can leverage predictive sales data and customer segments to optimize their distribution routes and delivery schedules. By anticipating demand for specific products, they can ensure efficient resource allocation and timely deliveries, improving customer satisfaction and operational efficiency.

CHAPTER 10

APPENDIX

10.1 Appendix A: Problem Statement Feasibility Assessment

The proposed system aims to develop an invoice intelligence and predictive analytics tool that enhances sales forecasting, invoice processing, and customer behavior analysis for businesses. To assess its feasibility:

- **Satisfiability:** The problem is satisfiable as it is feasible to implement a real-time sales forecasting and customer segmentation system using current technologies such as Python, TensorFlow, React, and MongoDB.
- **Complexity Classification:**
 - The system deals with data extraction, machine learning-based forecasting, and customer segmentation.
 - These tasks can be efficiently performed with existing machine learning frameworks and databases, solvable in polynomial time (**P-Type**), as no known NP-complete reductions or exponential search spaces are involved in the core functionality.
- **Mathematical Representation:** The system is modeled as:

$$S = \{I, P, O, F, D, A, R\}$$

where each element denotes inputs, processes, outputs, functions, database, actors, and rules respectively (as defined in the Mathematical Model section).

- **Modern Algebra Feasibility:** The modules and transitions can be visualized using a Finite State Machine (FSM) or function mappings:

$$f : I \rightarrow O \quad (\text{functions map inputs to outputs with defined rules})$$

making the system functionally complete and logically implementable.

Hence, the proposed problem is feasible, solvable in polynomial time, and does not fall under NP-Complete or NP-Hard categories.

10.2 Appendix B: Paper Publication Details

- **Title:** A Comparative Survey of Handwritten Text Recognition Techniques.
- **Name of Conference:** 2025 1st Hinweis Third International Conference on Advances in Information, Telecommunication and Computing (AITC).



**Hinweis Third International Conference on
Advances in Information, Telecommunication and
Computing (AITC)**

May 30-31, 2025, Virtual Conference
<http://aitc.thehinweis.com/2025>

PAPER REVIEW FORM	
Paper ID	AITC-2025_306
Paper Title	A Comparative Survey of Handwritten Text Recognition Techniques
Review Status	Accepted with Modification
Category(if accept)	Full Research Paper
Consolidated Content review comments	• Paper should strictly formatted according to the standard format • Adequate testing, results and its discussion are required. The paper could be revised with more results • More emphasis should be on related literature, problem statement, proposed system and its results & discussion

SI No	OVERALL RATING: (5= EXCELLENT, 1= POOR)	5	4	3	2	1
1.	Structure of the paper		X			
2.	Standard of the paper			X		
3.	Appropriateness of the title of the paper		X			
4.	Appropriateness of abstract as a description of the paper		X			
5.	Appropriateness of the research/study methods			X		
6.	Relevance and clarity of drawings, graph and table			X		
7.	Use and number of keywords / key phrases		X			
8.	Discussion and conclusion			X		
9.	Reference list, adequate and correctly cited			X		



AITC2025- May 30-31, 2025; Virtual Conference

<http://aitc.thehinweis.com/2025>

Figure 10.1: Publication Certificate

10.3 Appendix C: Plagiarism Report

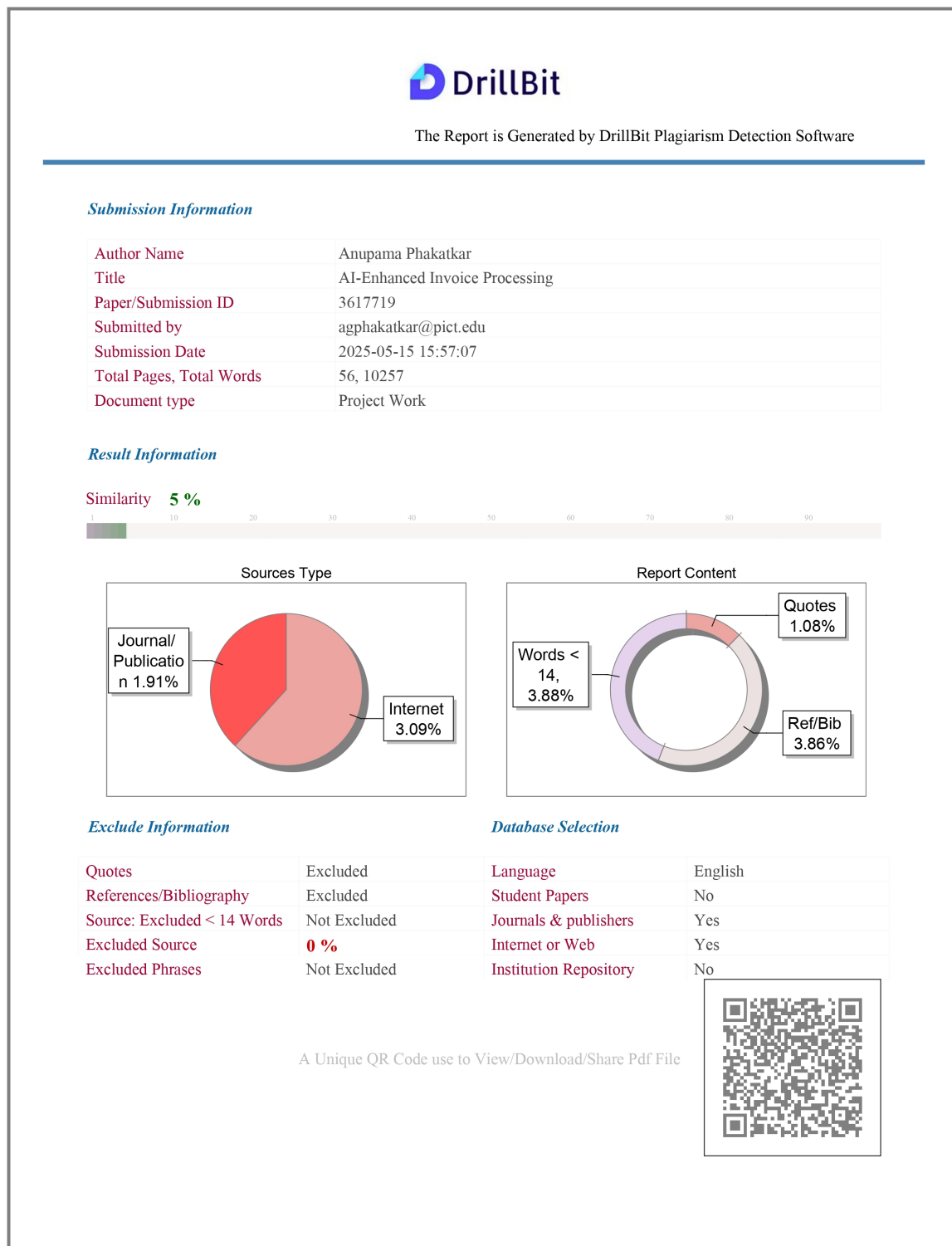


Figure 10.2: Publication Certificate

CHAPTER 11

REFERENCES

- [1] P. Sun, X. Yang, X. Zhao, and Z. Wang, “An Overview of Named Entity Recognition,” in *2018 International Conference on Asian Language Processing (IALP)*, Bandung, Indonesia, 2018, pp. 273–278, doi: 10.1109/IALP.2018.8629225.
- [2] D. R. Vedhaviyassh, R. Sudhan, G. Saranya, M. Safa, and D. Arun, “Comparative Analysis of EasyOCR and TesseractOCR for Automatic License Plate Recognition using Deep Learning Algorithm,” in *2022 6th International Conference on Electronics, Communication and Aerospace Technology*, Coimbatore, India, 2022, pp. 966–971, doi: 10.1109/ICECA55336.2022.10009215.
- [3] B. K. Pattanayak, A. K. Biswal, S. R. Laha, S. Pattnaik, B. B. Dash, and S. S. Patra, “A Novel Technique for Handwritten Text Recognition Using Easy OCR,” in *2023 International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*, Erode, India, 2023, pp. 1115–1119, doi: 10.1109/ICSSAS57918.2023.10331704.
- [4] V. Devisurya, S. A. Mohamed, A. Gavin, and A. Kamal, “Enhanced Number Plate Recognition for Restricted Area Access Control Using Deep Learning Models and EasyOCR Integration,” in *2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT)*, Vellore, India, 2024, pp. 1–6, doi: 10.1109/AI-IoT58432.2024.10574677.
- [5] S. Aftan and H. Shah, “A Survey on BERT and Its Applications,” in *2023 20th Learning and Technology Conference (L&T)*, Jeddah, Saudi Arabia, 2023, pp. 161–166, doi: 10.1109/LT58159.2023.10092289.
- [6] A. C. Berg, T. L. Berg, and J. Malik, “Shape Matching and Object Recognition Using Low Distortion Correspondence,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition*, San Diego, CA, June 2005.
- [7] D. Baviskar, S. Ahirrao, and K. Kotecha, “Multi-Layout Invoice Document Dataset (MIDD): A Dataset for Named Entity Recognition,” *Journal*, 2023.
- [8] T. E. de Campos and B. R. Babu, “Optical Character Recognition (OCR) Technology,” in *IIT Hyderabad Conference*, 2002.
- [9] L. Likforman-Sulem, A. Zahour, and B. Taconet, “Text Line Segmentation of Historical Documents: A Survey,” *International Journal on Document Analysis and Recognition*, 2007, pp. 123–138.
- [10] H. Singh and A. Sachan, “A Proposed Approach for Character Recognition Using Document Analysis with OCR,” in *2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS)*, 2018, pp. 190–195, doi: 10.1109/IC-CONS.2018.8663011.

LIST OF TABLES

Table	Illustration	Page No.
3.1	User Interfaces	10
3.2	Hardware Interfaces	11
3.3	Software Interfaces	11